

# PAN-INDIA LOGISTICS PLATFORM

Software Requirements Specification & Development Execution Plan

Hybrid Model: On-Demand Booking + Enterprise Logistics Management

|                |                                                                  |
|----------------|------------------------------------------------------------------|
| Document Type  | SRS + Development Plan                                           |
| Version        | 1.0                                                              |
| Date           | April 29, 2026                                                   |
| Classification | Confidential — Internal Use Only                                 |
| Timeline       | 40 Working Days (Phases 1–6)                                     |
| Tech Stack     | Flutter (Apps)   MERN (Web Portal)   Node.js + MongoDB (Backend) |
| Target Market  | Pan-India — B2C & B2B Logistics                                  |

# TABLE OF CONTENTS

---

- 1. Executive Summary
- 2. Project Scope & Objectives
- 3. System Architecture
- 4. Module Specifications
  - 4.1. Customer Mobile App (Flutter)
  - 4.2. Driver Mobile App (Flutter)
  - 4.3. Admin & Enterprise Web Portal (MERN)
  - 4.4. Backend API Server (Node.js + Express)
- 5. API Design & Contracts
- 6. Database Schema (MongoDB)
- 7. Third-Party Services & Integrations
- 8. Cost Estimation
- 9. 40-Day Development Plan (Day-by-Day)
- 10. Team Structure & Responsibilities
- 11. Risks & Mitigation
- 12. Testing Strategy
- 13. Deployment Plan
- 14. Future Roadmap

SECTION 1

# Executive Summary

This document is the official Software Requirements Specification (SRS) and Development Execution Plan for a Pan-India Hybrid Logistics Platform. It serves as the authoritative reference for the project manager, development team, QA team, and client stakeholders. All task scheduling, sprint planning, and deliverable tracking should be based on this document.

## What We Are Building

A full-stack, production-grade logistics platform combining two models:

| Porter-style (B2C)         | Om Logistics-style (B2B)           |
|----------------------------|------------------------------------|
| On-demand city delivery    | Enterprise supply chain management |
| Consumer-facing mobile app | Corporate dashboard with analytics |
| Live GPS tracking          | Multi-user enterprise accounts     |
| Instant price estimate     | Bulk bookings & invoice generation |
| Razorpay payment           | Monthly billing & audit trail      |

## Platforms Delivered

| Platform                  | Technology        | Primary Users                 | Key Purpose                            |
|---------------------------|-------------------|-------------------------------|----------------------------------------|
| Customer Mobile App       | Flutter           | End users / SMBs              | Book deliveries, track live            |
| Driver Mobile App         | Flutter           | Delivery drivers              | Accept orders, navigate, update status |
| Admin & Enterprise Portal | MERN Stack        | Ops team / Enterprise clients | Manage orders, analytics, invoicing    |
| Backend API Server        | Node.js + Express | All platforms                 | Business logic, data, real-time comms  |

SECTION 2

# Project Scope & Objectives

## In Scope

- Customer mobile app (Flutter) for Android & iOS
- Driver mobile app (Flutter) for Android
- Admin web portal (MERN) with analytics, order & fleet management
- Enterprise web portal (MERN) with multi-user accounts & invoicing
- RESTful API backend (Node.js + Express + MongoDB)
- Real-time GPS tracking via Socket.IO + Google Maps
- Razorpay payment gateway integration
- Firebase Cloud Messaging for push notifications
- JWT-based authentication across all platforms
- All vehicle types: 2-wheeler, auto, mini truck, medium truck, large truck
- Both intra-city and inter-city delivery flows
- Dynamic pricing engine based on distance, weight, vehicle type
- Invoice PDF generation for enterprise clients
- Role-based access control (Admin, Enterprise Admin, Enterprise User, Driver, Customer)
- Deployment on AWS EC2 with MongoDB Atlas

## Out of Scope (Phase 1)

- ✗ AI/ML-based route optimization (Future Phase)
- ✗ Warehouse Management System (Future Phase)
- ✗ Multi-language support beyond English & Hindi
- ✗ Insurance claim processing
- ✗ Full ERP integration

## Core Business Objectives

|             |                                                       |
|-------------|-------------------------------------------------------|
| Scalability | Architecture must support 10,000+ concurrent bookings |
| Reliability | 99.5% API uptime; graceful failover                   |
| Real-time   | GPS updates every 3–5 seconds; <200ms latency         |
| Security    | JWT auth, encrypted payments, HTTPS-only              |
| Modularity  | Each service independently deployable and testable    |

SECTION 3

System Architecture

High-Level Architecture

The platform follows a three-tier microservice-style architecture: (1) Client Tier — Flutter mobile apps and React.js web portals (2) Application Tier — Node.js REST API with Socket.IO real-time layer (3) Data Tier — MongoDB Atlas cloud database + AWS S3 for file storage All external services (Maps, Payments, Notifications) are integrated at the Application Tier via their respective SDKs/APIs. Internal communication between services uses event-driven patterns.

|                   |                              |                              |                        |
|-------------------|------------------------------|------------------------------|------------------------|
| CLIENT TIER       | Flutter Customer App         | Flutter Driver App           | React Admin Portal     |
|                   | ↓ <i>HTTPS/REST</i>          | ↓ <i>HTTPS/REST + Socket</i> | ↓ <i>HTTPS/REST</i>    |
| APPLICATION TIER  | Node.js API Gateway          | Auth Service (JWT)           | Booking Service        |
|                   | Tracking Service (Socket.IO) | Payment Service              | Notification Service   |
|                   | ↓ <i>Mongoose ODM</i>        | ↓ <i>Mongoose ODM</i>        | ↓ <i>AWS S3 SDK</i>    |
| DATA TIER         | MongoDB Atlas                | AWS S3 Storage               | Redis Cache (optional) |
| EXTERNAL SERVICES | Google Maps API              | Razorpay Gateway             | Firebase FCM           |

Real-Time Communication Flow

Driver app sends GPS coordinates every 3–5 seconds via Socket.IO. The tracking service receives these events, stores the last known position, and broadcasts to the customer's app room. The customer app renders the moving pin on Google Maps in real time. Order status changes also propagate via sockets to both driver and customer simultaneously.

## SECTION 4

## Module Specifications

---

### 4.1 Customer Mobile App (Flutter)

**Onboarding & Auth:** Phone/Email signup, OTP verification, JWT login, profile setup

**Home Screen:** City selector, quick booking CTA, active orders widget, promo banners

**Booking Flow:** Pickup location → Drop location → Select vehicle → Goods type → Weight → Price estimate → Confirm & Pay

**Live Tracking:** Google Maps screen, moving driver pin updated via Socket, ETA, status timeline

**Booking History:** Paginated list of past orders, status, price; filter by date/status

**Invoice & Receipts:** View/download PDF invoice per booking; share via WhatsApp/email

**Enterprise Upgrade:** Multi-user account setup, sub-user invites, centralized billing

**Notifications:** FCM push: order confirmed, driver assigned, out for delivery, delivered

**Support:** In-app ticket creation, chat thread, contact center link

**Profile & Settings:** Edit profile, saved addresses, preferred payment method, logout

### 4.2 Driver Mobile App (Flutter)

**Auth & Onboarding:** Phone login, document upload (license, RC), admin verification before activation

**Order Queue:** Available orders list sorted by proximity; accept/reject with timer

**Active Order:** Navigation to pickup → navigation to drop via Google Maps turn-by-turn

**GPS Broadcasting:** Background GPS service broadcasting coordinates via Socket.IO every 4 seconds

**Status Updates:** Driver taps: Arrived → Picked Up → In Transit → Delivered + OTP/photo POD

**Earnings Dashboard:** Daily/weekly/monthly earnings, trip count, ratings received

**Documents & Profile:** View submitted documents, update vehicle info, bank details for payout

**Offline Handling:** Queue GPS updates locally; sync when connection restored

### 4.3 Admin & Enterprise Web Portal (MERN)

#### Admin Panel Features:

**Master Dashboard:** KPIs: total orders today, active deliveries, revenue, driver utilization, top routes

**Order Management:** Full orders table with filters (status, city, date, vehicle type), manual reassign driver

**Driver Management:** Onboard/suspend drivers, view live location on map, performance scores, documents

**Vehicle Management:** Vehicle inventory by type, availability status, maintenance alerts

**User Management:** Customer accounts, enterprise accounts, ban/unban, KYC status

**Pricing Engine:** Set base fares, per-km rates, surcharges per vehicle type and city

**Live Fleet Map:** Real-time map of all active drivers, clustered markers, click for driver detail

**Analytics & Reports:** Revenue by city/date/vehicle, delivery success rate, heatmaps, export CSV

**Notification Blast:** Send push to all customers, drivers, or specific segments

**Content Management:** Manage promo banners, FAQs, app notifications

#### Enterprise Portal Features:

**Enterprise Dashboard:** Company-level KPIs: monthly spend, active shipments, top destinations

**Bulk Booking:** CSV upload or manual multi-row booking form; validate before submit

**Order Tracking:** Track all company orders in one view, filter by user/status/date

**User Management:** Add/remove sub-users, assign roles (viewer/booker/admin), set spending limits

**Invoice & Billing:** Auto-generated monthly PDF invoices with itemized breakdowns; download/email

**GST & Tax Reports:** GST-compliant billing, TDS tracking, export for accounting software

**Contract Pricing:** Custom rate cards negotiated per enterprise account

**API Access:** Enterprise API key for direct system integration (webhook + REST)

## 4.4 Backend API Server (Node.js + Express)

**Auth Service:** JWT issue/verify, refresh tokens, OTP via SMS (MSG91/Twilio), role guards

**Booking Service:** Create, update, cancel bookings; driver allocation algorithm; price calculation

**Tracking Service:** Socket.IO rooms per booking; accept driver GPS events; broadcast to customer

**Payment Service:** Razorpay order creation, webhook verification, refund handling, transaction logs

**Notification Service:** FCM push via Firebase Admin SDK; email via SendGrid; SMS via MSG91

**Invoice Service:** Generate PDF invoices using PDFKit; store on S3; email to enterprise

**Analytics Service:** Aggregation pipelines on MongoDB for dashboard KPIs and report exports

**Admin Service:** CRUD for drivers, vehicles, users, pricing; audit logging



## SECTION 5

## API Design & Contracts

All APIs are RESTful over HTTPS. Authentication via Bearer JWT token in Authorization header. All responses follow the format: { success: bool, data: {}, message: string, statusCode: int }. API versioning: /api/v1/...

### Auth APIs

| Method | Endpoint                   | Description                        |
|--------|----------------------------|------------------------------------|
| POST   | /api/v1/auth/send-otp      | Send OTP to phone/email            |
| POST   | /api/v1/auth/verify-otp    | Verify OTP and issue JWT           |
| POST   | /api/v1/auth/register      | Complete user profile registration |
| POST   | /api/v1/auth/login         | Email/password login               |
| GET    | /api/v1/auth/profile       | Get authenticated user profile     |
| PUT    | /api/v1/auth/profile       | Update profile details             |
| POST   | /api/v1/auth/refresh-token | Issue new access token             |
| POST   | /api/v1/auth/logout        | Invalidate refresh token           |

### Booking APIs

| Method | Endpoint                     | Description                             |
|--------|------------------------------|-----------------------------------------|
| POST   | /api/v1/booking/estimate     | Get price estimate before booking       |
| POST   | /api/v1/booking/create       | Create new booking                      |
| GET    | /api/v1/booking/:id          | Get booking details by ID               |
| GET    | /api/v1/booking/user/:userId | List all bookings for a user            |
| PUT    | /api/v1/booking/:id/cancel   | Cancel booking                          |
| GET    | /api/v1/booking/:id/track    | Get current driver location for booking |

### Driver APIs

| Method | Endpoint                         | Description                          |
|--------|----------------------------------|--------------------------------------|
| GET    | /api/v1/driver/available-orders  | List nearby available orders         |
| POST   | /api/v1/driver/accept/:bookingId | Accept an order                      |
| POST   | /api/v1/driver/reject/:bookingId | Reject an order                      |
| PUT    | /api/v1/driver/status            | Update driver availability           |
| POST   | /api/v1/driver/pod/:bookingId    | Upload proof of delivery (photo/OTP) |

| Method | Endpoint                | Description          |
|--------|-------------------------|----------------------|
| GET    | /api/v1/driver/earnings | Get earnings summary |

## Payment APIs

| Method | Endpoint                     | Description                           |
|--------|------------------------------|---------------------------------------|
| POST   | /api/v1/payment/create-order | Create Razorpay order for booking     |
| POST   | /api/v1/payment/verify       | Verify Razorpay payment signature     |
| POST   | /api/v1/payment/refund       | Initiate refund for cancelled booking |
| GET    | /api/v1/payment/history      | Get payment history for user          |

## Enterprise APIs

| Method | Endpoint                            | Description                       |
|--------|-------------------------------------|-----------------------------------|
| POST   | /api/v1/enterprise/create           | Register a new enterprise account |
| GET    | /api/v1/enterprise/dashboard        | Get enterprise KPI dashboard data |
| POST   | /api/v1/enterprise/bulk-booking     | Submit bulk booking via CSV/JSON  |
| POST   | /api/v1/enterprise/users/invite     | Invite sub-user to enterprise     |
| GET    | /api/v1/enterprise/invoices         | List all invoices for enterprise  |
| GET    | /api/v1/enterprise/invoices/:id/pdf | Download invoice PDF              |

## Admin APIs

| Method | Endpoint                        | Description                     |
|--------|---------------------------------|---------------------------------|
| GET    | /api/v1/admin/orders            | List all orders with filters    |
| PUT    | /api/v1/admin/orders/:id/assign | Manually assign driver to order |
| GET    | /api/v1/admin/drivers           | List all drivers with status    |
| PUT    | /api/v1/admin/drivers/:id       | Update driver status / suspend  |
| GET    | /api/v1/admin/analytics         | Get platform analytics data     |
| PUT    | /api/v1/admin/pricing           | Update pricing configuration    |

## SECTION 6

## Database Schema (MongoDB)

### Users Collection

| Field                        | Type             | Description                                                    |
|------------------------------|------------------|----------------------------------------------------------------|
| <b>_id</b>                   | <i>ObjectId</i>  | Primary key                                                    |
| <b>name</b>                  | <i>String</i>    | Full name                                                      |
| <b>phone</b>                 | <i>String</i>    | Mobile (unique)                                                |
| <b>email</b>                 | <i>String</i>    | Email (unique)                                                 |
| <b>passwordHash</b>          | <i>String</i>    | bcrypt hashed password                                         |
| <b>role</b>                  | <i>Enum</i>      | customer   driver   admin   enterprise_admin   enterprise_user |
| <b>isVerified</b>            | <i>Boolean</i>   | OTP/KYC verified                                               |
| <b>fcmToken</b>              | <i>String</i>    | Firestore push token                                           |
| <b>savedAddresses</b>        | <i>[Address]</i> | Array of saved pickup/drop locations                           |
| <b>createdAt / updatedAt</b> | <i>Date</i>      | Timestamps                                                     |

### Orders Collection

| Field                  | Type                         | Description                                                         |
|------------------------|------------------------------|---------------------------------------------------------------------|
| <b>_id</b>             | <i>ObjectId</i>              | Primary key                                                         |
| <b>customerId</b>      | <i>ObjectId → Users</i>      | Reference to customer                                               |
| <b>driverId</b>        | <i>ObjectId → Drivers</i>    | Assigned driver (null until assigned)                               |
| <b>enterpriseId</b>    | <i>ObjectId → Enterprise</i> | Null for B2C orders                                                 |
| <b>pickupLocation</b>  | <i>GeoJSON Point</i>         | { type, coordinates, address }                                      |
| <b>dropLocation</b>    | <i>GeoJSON Point</i>         | { type, coordinates, address }                                      |
| <b>waypoints</b>       | <i>[GeoJSON Point]</i>       | Optional intermediate stops                                         |
| <b>vehicleType</b>     | <i>Enum</i>                  | bike   auto   mini_truck   medium_truck   large_truck               |
| <b>goodsType</b>       | <i>String</i>                | Category of goods being transported                                 |
| <b>weightKg</b>        | <i>Number</i>                | Declared weight in kg                                               |
| <b>distanceKm</b>      | <i>Number</i>                | Calculated route distance                                           |
| <b>price</b>           | <i>Number</i>                | Final quoted/charged price                                          |
| <b>status</b>          | <i>Enum</i>                  | pending   accepted   picked_up   in_transit   delivered   cancelled |
| <b>paymentStatus</b>   | <i>Enum</i>                  | unpaid   paid   refunded                                            |
| <b>razorpayOrderId</b> | <i>String</i>                | Razorpay order reference                                            |
| <b>otp</b>             | <i>String</i>                | Delivery confirmation OTP                                           |
| <b>podImageUrl</b>     | <i>String</i>                | S3 URL for proof of delivery                                        |

| Field                        | Type                 | Description                          |
|------------------------------|----------------------|--------------------------------------|
| <b>timeline</b>              | <i>[StatusEvent]</i> | Array of { status, timestamp, note } |
| <b>createdAt / updatedAt</b> | <i>Date</i>          | Timestamps                           |

## Drivers Collection

| Field                  | Type                    | Description                                           |
|------------------------|-------------------------|-------------------------------------------------------|
| <b>_id</b>             | <i>ObjectId</i>         | Primary key                                           |
| <b>userId</b>          | <i>ObjectId → Users</i> | Reference to user record                              |
| <b>vehicleType</b>     | <i>Enum</i>             | bike   auto   mini_truck   medium_truck   large_truck |
| <b>vehicleNumber</b>   | <i>String</i>           | Registration number                                   |
| <b>licenseNumber</b>   | <i>String</i>           | Driving license                                       |
| <b>currentLocation</b> | <i>GeoJSON Point</i>    | Last known GPS coordinates                            |
| <b>isAvailable</b>     | <i>Boolean</i>          | Online/offline toggle                                 |
| <b>rating</b>          | <i>Number</i>           | Average rating (1–5)                                  |
| <b>totalTrips</b>      | <i>Number</i>           | Lifetime trip count                                   |
| <b>totalEarnings</b>   | <i>Number</i>           | Cumulative earnings                                   |
| <b>documents</b>       | <i>{}</i>               | { licenseUrl, rcUrl, aadhaarUrl, photoUrl }           |
| <b>isApproved</b>      | <i>Boolean</i>          | Admin approved for operations                         |
| <b>createdAt</b>       | <i>Date</i>             | Onboarding date                                       |

## Payments Collection

| Field                    | Type                     | Description                            |
|--------------------------|--------------------------|----------------------------------------|
| <b>_id</b>               | <i>ObjectId</i>          | Primary key                            |
| <b>orderId</b>           | <i>ObjectId → Orders</i> | Reference to order                     |
| <b>userId</b>            | <i>ObjectId → Users</i>  | Customer who paid                      |
| <b>razorpayOrderId</b>   | <i>String</i>            | Razorpay order ID                      |
| <b>razorpayPaymentId</b> | <i>String</i>            | Razorpay payment ID                    |
| <b>razorpaySignature</b> | <i>String</i>            | Webhook signature for verification     |
| <b>amount</b>            | <i>Number</i>            | Amount in INR paise                    |
| <b>currency</b>          | <i>String</i>            | INR                                    |
| <b>status</b>            | <i>Enum</i>              | created   captured   failed   refunded |
| <b>refundId</b>          | <i>String</i>            | Razorpay refund ID (if applicable)     |
| <b>createdAt</b>         | <i>Date</i>              | Payment timestamp                      |

## Enterprise Collection

| Field                  | Type                         | Description                             |
|------------------------|------------------------------|-----------------------------------------|
| <b>_id</b>             | <i>ObjectId</i>              | Primary key                             |
| <b>companyName</b>     | <i>String</i>                | Registered company name                 |
| <b>gstIn</b>           | <i>String</i>                | GST Identification Number               |
| <b>adminUserId</b>     | <i>ObjectId → Users</i>      | Primary admin user                      |
| <b>subUsers</b>        | <i>[ObjectId]</i>            | Array of enterprise user IDs            |
| <b>contractPricing</b> | <i>{}</i>                    | Custom rate card overrides              |
| <b>billingCycle</b>    | <i>Enum</i>                  | monthly   fortnightly                   |
| <b>billingEmail</b>    | <i>String</i>                | Invoice delivery email                  |
| <b>creditLimit</b>     | <i>Number</i>                | Maximum credit before mandatory payment |
| <b>invoices</b>        | <i>[ObjectId → Invoices]</i> | Reference to invoice documents          |
| <b>isActive</b>        | <i>Boolean</i>               | Account status                          |
| <b>createdAt</b>       | <i>Date</i>                  | Account creation date                   |

## SECTION 7

## Third-Party Services & Integrations

| Service              | Purpose & Notes                                                                                 | Category       | Est. Cost                    |
|----------------------|-------------------------------------------------------------------------------------------------|----------------|------------------------------|
| Google Maps Platform | Maps SDK (Flutter), Directions API, Distance Matrix API, Places Autocomplete API, Geocoding API | Maps & Routing | Usage-based (~\$50–\$200/mo) |
| Razorpay             | Payment Gateway: orders, webhooks, refunds. Supports UPI, cards, netbanking, wallets            | Payments       | 2% per transaction           |
| Firebase             | Firebase Cloud Messaging (FCM) for push notifications on Android & iOS                          | Notifications  | Free                         |
| MSG91 / Twilio       | SMS OTP delivery for phone verification                                                         | SMS OTP        | ~Rs 0.15 per SMS             |
| SendGrid / AWS SES   | Transactional emails: registration, booking confirmation, invoice delivery                      | Email          | \$0–\$15/mo                  |
| AWS EC2              | Backend server hosting (t3.medium recommended for production)                                   | Hosting        | \$30–\$60/mo                 |
| MongoDB Atlas        | Fully managed cloud MongoDB (M10 cluster for production)                                        | Database       | \$57/mo                      |
| AWS S3               | File storage: POD images, invoice PDFs, driver documents                                        | Storage        | \$5–\$20/mo                  |
| PDFKit (Node.js)     | Server-side PDF generation for invoices and receipts                                            | PDF Generation | Free (OSS)                   |
| Socket.IO            | Real-time WebSocket layer for GPS tracking and status updates                                   | Real-time      | Free (OSS)                   |
| JWT (jsonwebtoken)   | Access & refresh token generation/verification                                                  | Auth           | Free (OSS)                   |
| Mongoose ODM         | MongoDB object modeling for Node.js                                                             | DB Layer       | Free (OSS)                   |

SECTION 8

Cost Estimation

| Service / Resource   | Tier          | Monthly Est. (INR)  | Notes                           |
|----------------------|---------------|---------------------|---------------------------------|
| MongoDB Atlas        | M10 Cluster   | ~₹4,700             | Dedicated cluster, auto-backups |
| AWS EC2 (Backend)    | t3.medium     | ~₹3,500             | 2 vCPU, 4GB RAM                 |
| AWS S3 (Storage)     | Standard      | ~₹800               | Based on ~10GB/mo               |
| Google Maps Platform | Pay-as-you-go | ~₹4,000–₹16,000     | Scales with booking volume      |
| Razorpay             | Standard      | 2% per txn          | No monthly fee                  |
| Firebase FCM         | Spark Plan    | ₹0                  | Free up to high volume          |
| MSG91 (SMS OTP)      | Standard      | ~₹1,500             | ~10,000 OTPs/mo                 |
| SendGrid (Email)     | Essentials    | ~₹500               | Up to 50K emails/mo             |
| Domain + SSL         | Annual        | ~₹1,000/yr          | Route53 + ACM                   |
| Total (Approx.)      |               | ~₹16,000–₹28,000/mo | Scales with usage               |

Note: Google Maps API costs are the most variable. To control costs, implement aggressive caching of route results, use Distance Matrix only for final booking confirmation (not on every keystroke), and set billing alerts on Google Cloud Console.

SECTION 9

40-Day Development Plan (Day-by-Day)

Phase 1: Foundation & Architecture (Days 1–7)

| Day   | Team     | Tasks                                                                                                     |
|-------|----------|-----------------------------------------------------------------------------------------------------------|
| Day 1 | All devs | Project kickoff, Git repo setup, folder structure, coding standards, branch strategy (main/dev/feature/*) |
| Day 1 | Backend  | Node.js project init: Express + Mongoose + dotenv + helmet + cors + morgan. MongoDB Atlas cluster setup   |
| Day 2 | Backend  | Auth Service: JWT middleware, OTP send (MSG91), OTP verify, register, login endpoints                     |
| Day 2 | MERN     | React app init: Vite + TailwindCSS + React Router + Axios. Admin login page skeleton                      |
| Day 2 | Flutter  | Flutter project init: folder structure, dio, riverpod/bloc, go_router, google_maps_flutter packages       |
| Day 3 | Backend  | User model, Driver model, MongoDB indexes (geospatial 2dsphere on locations). Seed script                 |
| Day 3 | Flutter  | Splash screen, onboarding flow, phone entry, OTP input screen (customer app)                              |
| Day 4 | Backend  | Booking model, Orders collection, Payments model. Basic CRUD scaffolding                                  |
| Day 4 | Flutter  | Driver app: login screen, OTP, home/availability toggle skeleton                                          |
| Day 5 | Backend  | Booking Service: create booking, price estimate logic, status state machine                               |
| Day 5 | MERN     | Admin: sidebar layout, dashboard shell, API interceptor with JWT                                          |
| Day 6 | All      | Integration test: Auth flow end-to-end (register → login → profile). Fix blockers                         |
| Day 7 | All      | Code review, refactor, write base unit tests for auth service. Deploy to dev server                       |

Phase 2: Booking Flow & Core UI (Days 8–14)

| Day    | Team    | Tasks                                                                                             |
|--------|---------|---------------------------------------------------------------------------------------------------|
| Day 8  | Flutter | Customer: Google Maps screen, location picker (pickup), Places Autocomplete integration           |
| Day 8  | Backend | Booking estimate API: integrate Distance Matrix API, pricing formula by vehicle & distance        |
| Day 9  | Flutter | Customer: drop location picker, vehicle type selector (scrollable cards with icons)               |
| Day 9  | MERN    | Admin: Orders table with filters (status, date, city), pagination, status badge components        |
| Day 10 | Flutter | Customer: Goods type, weight input, price estimate screen, booking confirmation screen            |
| Day 10 | Backend | Booking create API, driver allocation algorithm (nearest available by vehicle type)               |
| Day 11 | Flutter | Customer: Booking history list, booking detail screen, status timeline component                  |
| Day 11 | MERN    | Admin: Driver management table, add/edit driver form, vehicle management CRUD                     |
| Day 12 | Flutter | Driver app: Available orders list, order detail card, accept/reject with timer                    |
| Day 12 | Backend | Driver order assignment push via FCM; order accept/reject endpoints                               |
| Day 13 | Flutter | Driver app: Active order screen — navigation to pickup, navigation to drop, status update buttons |



| Day    | Team        | Tasks                                                                                        |
|--------|-------------|----------------------------------------------------------------------------------------------|
| Day 13 | <b>MERN</b> | Admin: User management, enterprise account listing, basic analytics KPI cards                |
| Day 14 | <b>All</b>  | Integration test: Full booking flow (create → assign → accept). Bug fixes. Mid-sprint review |

### Phase 3: Real-Time Tracking & Payments (Days 15–21)

| Day    | Team           | Tasks                                                                                                |
|--------|----------------|------------------------------------------------------------------------------------------------------|
| Day 15 | <b>Backend</b> | Socket.IO server setup: rooms per bookingId, driver_location_update event, broadcast to customer     |
| Day 15 | <b>Flutter</b> | Driver app: Background location service (geolocator), Socket.IO emit GPS every 4s                    |
| Day 16 | <b>Flutter</b> | Customer: Live tracking screen — Google Maps, listen Socket events, animate driver marker            |
| Day 16 | <b>Backend</b> | Tracking Service: store last position in Redis/memory, ETA calculation using Directions API          |
| Day 17 | <b>Flutter</b> | Customer: ETA display, driver info card (name, vehicle, rating, call button)                         |
| Day 17 | <b>MERN</b>    | Admin: Live fleet map — all active drivers on Google Maps, clustered markers, click for info         |
| Day 18 | <b>Backend</b> | Razorpay integration: create order, payment capture webhook, signature verification                  |
| Day 18 | <b>Flutter</b> | Customer: Razorpay Flutter SDK, payment screen, success/failure handling, update booking             |
| Day 19 | <b>Backend</b> | Refund API, payment history API, Razorpay webhook handler with retry logic                           |
| Day 19 | <b>Flutter</b> | Driver app: OTP confirmation for delivery, photo upload for POD (S3 via signed URL)                  |
| Day 20 | <b>Backend</b> | Notification Service: FCM push for all order status transitions, batch notification for admin blasts |
| Day 20 | <b>Flutter</b> | Both apps: FCM integration, notification tap → deep link to relevant screen                          |
| Day 21 | <b>All</b>     | Full flow integration test: booking → tracking → payment → delivery confirmation. QA cycle           |

### Phase 4: Enterprise Features & Invoicing (Days 22–28)

| Day    | Team           | Tasks                                                                                               |
|--------|----------------|-----------------------------------------------------------------------------------------------------|
| Day 22 | <b>Backend</b> | Enterprise model, create enterprise account, invite sub-users, role-based guards middleware         |
| Day 22 | <b>MERN</b>    | Enterprise portal: registration flow, dashboard shell, sub-user invite system                       |
| Day 23 | <b>Backend</b> | Bulk booking API: parse CSV, validate rows, batch create orders, return summary                     |
| Day 23 | <b>MERN</b>    | Enterprise: Bulk booking UI — CSV upload, manual multi-row form, validation table, confirm          |
| Day 24 | <b>Backend</b> | Invoice Service: aggregate monthly bookings, calculate totals, generate PDF via PDFKit, store on S3 |
| Day 24 | <b>MERN</b>    | Enterprise: Invoices list, invoice detail view, download PDF button, send to billing email          |
| Day 25 | <b>Backend</b> | GST-compliant invoice format: GSTIN, HSN code, CGST/SGST/IGST breakdown                             |

| Day    | Team           | Tasks                                                                                                |
|--------|----------------|------------------------------------------------------------------------------------------------------|
| Day 25 | <b>MERN</b>    | Enterprise: Analytics dashboard — monthly spend chart, top routes, delivery success rate, CSV export |
| Day 26 | <b>Backend</b> | Contract pricing: custom rate card per enterprise, override standard pricing logic                   |
| Day 26 | <b>Flutter</b> | Customer app: Enterprise upgrade flow, multi-user management, centralized billing view               |
| Day 27 | <b>MERN</b>    | Admin: Enterprise account management, approve/reject, set credit limits, manual invoice trigger      |
| Day 27 | <b>Backend</b> | Admin analytics API: revenue by city/vehicle/date aggregation pipelines                              |
| Day 28 | <b>All</b>     | Enterprise feature integration test, invoice generation test, QA review                              |

## Phase 5: Polish, Advanced Features & Testing (Days 29–35)

| Day    | Team           | Tasks                                                                                                           |
|--------|----------------|-----------------------------------------------------------------------------------------------------------------|
| Day 29 | <b>Flutter</b> | Customer: Saved addresses, profile edit, notification preferences, in-app support ticket                        |
| Day 29 | <b>Backend</b> | Support ticket model, create ticket, admin reply, email notification on reply                                   |
| Day 30 | <b>MERN</b>    | Admin: Content management (banners, FAQs), notification blast tool, pricing engine UI                           |
| Day 30 | <b>Backend</b> | Admin pricing API: per-city, per-vehicle rates, surge pricing toggle, effective from/to dates                   |
| Day 31 | <b>Flutter</b> | Driver app: Earnings dashboard (daily/weekly/monthly), trip history, rating display                             |
| Day 31 | <b>Backend</b> | Earnings calculation service, payout history model, bank detail management for drivers                          |
| Day 32 | <b>All</b>     | Performance testing: API load test with k6 (500 concurrent), MongoDB query optimization, add indexes            |
| Day 33 | <b>All</b>     | Security audit: JWT secret rotation, rate limiting (express-rate-limit), input sanitization (express-validator) |
| Day 34 | <b>QA</b>      | End-to-end test all flows: customer booking, tracking, payment, driver acceptance, POD, enterprise bulk         |
| Day 35 | <b>All</b>     | Bug fixes from QA report, regression testing, code freeze for non-critical features                             |

## Phase 6: Deployment & Go-Live (Days 36–40)

| Day    | Team           | Tasks                                                                                                  |
|--------|----------------|--------------------------------------------------------------------------------------------------------|
| Day 36 | <b>Backend</b> | Production environment setup: AWS EC2, environment variables, PM2 process manager, Nginx reverse proxy |
| Day 36 | <b>Backend</b> | MongoDB Atlas production cluster, connection string, IP whitelist, daily backup policy                 |
| Day 37 | <b>Backend</b> | SSL certificate (ACM/Certbot), HTTPS enforcement, CORS production config, API rate limiting            |
| Day 37 | <b>MERN</b>    | Admin/Enterprise portal: production build (Vite), S3 static hosting or Vercel deployment, env config   |
| Day 38 | <b>Flutter</b> | Customer app: production keystore signing, Google Maps API key restrictions, build APK/IPA             |
| Day 38 | <b>Flutter</b> | Driver app: production build, GPS background mode permissions, battery optimization config             |

| Day    | Team | Tasks                                                                                                         |
|--------|------|---------------------------------------------------------------------------------------------------------------|
| Day 39 | All  | UAT (User Acceptance Testing) with stakeholders, smoke test all platforms on production                       |
| Day 40 | All  | Final bug fixes from UAT, monitoring setup (PM2 logs, AWS CloudWatch alerts), go-live, handover documentation |

SECTION 10

Team Structure & Responsibilities

| Role                        | Count | Primary Responsibilities                                                               | Phases     |
|-----------------------------|-------|----------------------------------------------------------------------------------------|------------|
| Flutter Developer (Senior)  | 1     | Customer app architecture, Google Maps, real-time socket, payment SDK, app store build | All phases |
| Flutter Developer (Junior)  | 1     | Driver app, OTP flow, GPS service, POD upload, earnings screens                        | Phase 2–5  |
| Backend Developer (Node.js) | 1     | API server, Socket.IO, Razorpay, Auth, Driver allocation logic, deployment             | All phases |
| MERN Developer              | 1     | Admin portal, Enterprise portal, React components, analytics charts                    | Phase 1–5  |
| QA Engineer                 | 1     | Test plans, manual testing all flows, API testing (Postman), regression                | Phase 5–6  |
| Project Manager             | 1     | Daily standups, task tracking, client communication, risk management                   | All phases |

Daily Standup Agenda (Recommended)

- 1. What did you complete yesterday?
- 2. What will you work on today?
- 3. Any blockers or dependencies?
- 4. Are you on track with the phase milestone?

## SECTION 11

## Risks & Mitigation

| Severity | Risk                                    | Mitigation Strategy                                                                                                         |
|----------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| HIGH     | Google Maps API cost overrun            | Implement caching for routes, restrict API key by domain/bundle ID, set \$50/day billing alert on GCP Console               |
| HIGH     | 40-day timeline is tight for full scope | Prioritize MVP: booking + tracking + payment. Enterprise features are Phase 5 bonus. Cut scope, not quality                 |
| HIGH     | Real-time GPS accuracy issues           | Use geolocator with high accuracy mode, apply GPS smoothing (Kalman filter or simple averaging), handle GPS loss gracefully |
| MEDIUM   | Razorpay payment webhook failure        | Implement idempotent webhook handler, retry queue, manual verification endpoint for fallback                                |
| MEDIUM   | Driver allocation edge cases            | Handle no-driver-available gracefully, implement retry after 2 min, admin manual assign as fallback                         |
| MEDIUM   | Flutter background GPS on iOS           | Use background_locator_2 or geolocator with background mode; test extensively on iOS 16+                                    |
| LOW      | MongoDB Atlas connectivity from EC2     | Use VPC peering or IP whitelist; connection pooling via Mongoose to limit connections                                       |
| LOW      | FCM token staleness                     | Refresh FCM token on each app launch, update in DB; handle SendRegistrationTokensRequest errors                             |

SECTION 12

Testing Strategy

| Test Type           | Owner        | Scope                                                                                       | Timeline       |
|---------------------|--------------|---------------------------------------------------------------------------------------------|----------------|
| Unit Testing        | Backend      | Jest — Auth service, Pricing logic, Invoice generation                                      | Day 7, 14, 21  |
| Integration Testing | Backend      | Supertest — Full API flows: booking lifecycle, payment webhook                              | Day 14, 21, 28 |
| API Testing         | QA           | Postman Collection with environment variables for all 40+ endpoints                         | Ongoing        |
| UI Testing          | QA           | Manual: all screens on Android emulator + physical device                                   | Phase 5        |
| Real-time Testing   | QA           | Simulate driver GPS movement, verify customer sees live update <5s delay                    | Day 21         |
| Payment Testing     | QA + Backend | Razorpay test mode: success, failure, webhook retry scenarios                               | Day 19–20      |
| Load Testing        | Backend      | k6 scripts: 500 concurrent booking creates, measure p95 latency target <500ms               | Day 32         |
| Security Testing    | Backend      | OWASP basics: SQL injection (not applicable but NoSQL injection), JWT expiry, rate limiting | Day 33         |
| UAT                 | PM + Client  | Stakeholder walkthroughs of all critical paths on production environment                    | Day 39         |
| Regression Testing  | QA           | Full smoke test after every major merge to main branch                                      | Day 35, 39     |

SECTION 13

# Deployment Plan

## Infrastructure Overview

- AWS EC2 (t3.medium):** Node.js API server + Socket.IO, managed by PM2, behind Nginx reverse proxy
- MongoDB Atlas (M10):** Production database cluster with auto-backup, VPC peering recommended
- AWS S3:** Static file storage: POD images, invoices, driver documents. CloudFront CDN optional
- Vercel / AWS S3 + CloudFront:** Admin & Enterprise React portal static hosting
- Google Play Store:** Customer & Driver Android APKs
- Apple App Store:** Customer iOS IPA (requires Apple Developer account)

## Environment Strategy

| Environment | Purpose                    | Branch    | URL Pattern                |
|-------------|----------------------------|-----------|----------------------------|
| Development | Active feature development | feature/* | dev.api.yourdomain.com     |
| Staging     | QA testing, UAT            | develop   | staging.api.yourdomain.com |
| Production  | Live users                 | main      | api.yourdomain.com         |

## CI/CD Pipeline (Recommended)

- GitHub Actions: on push to develop → run Jest tests → deploy to staging EC2
- On merge to main → run full test suite → deploy to production EC2 via SSH
- Versioned releases with Git tags (v1.0.0, v1.1.0)
- Flutter: manual build + upload to internal testing track (Google Play Console)

## SECTION 14

## Future Roadmap (Post Phase 1)

---

### Phase 2 (Month 2–3)

- AI-based driver allocation (ML model using historical demand patterns)
- Route optimization engine (multi-stop delivery optimization)
- Customer loyalty program and referral system
- Multi-language support (Hindi, Tamil, Telugu, Bengali)
- In-app chat between customer and driver

### Phase 3 (Month 4–6)

- Warehouse Management System integration
- Fleet management module (maintenance tracking, fuel logs)
- Advanced analytics and BI dashboard with Power BI/Metabase
- Third-party logistics (3PL) partner onboarding
- Cross-border shipping (Nepal, Bangladesh) expansion

### Phase 4 (Month 6+)

- Insurance integration for high-value goods
- Customs documentation for international shipments
- Driver incentive gamification (badges, bonuses)
- IoT sensor integration for temperature-controlled logistics
- Public API marketplace for enterprise integrations (like Porter Enterprise API)



## Document Sign-Off

| Role                  | Name | Signature | Date |
|-----------------------|------|-----------|------|
| Project Manager       |      |           |      |
| Lead Developer        |      |           |      |
| Client Representative |      |           |      |
| QA Lead               |      |           |      |

This document is version 1.0. All changes to scope, timeline, or technical decisions must be documented as versioned amendments and signed off by the Project Manager and Client Representative before implementation.